面向对象程序设计基础

面向对象基本概念

- 面向对象（Object Oriented），能以真实世界中的方式在程序中表示物体（Object）
- 程序中表示的物体就是对象
- 物体有特征和行为，对象有属性和方法
- 对象通过类创建（实例化）
- 类（Class）：设计图
- 对象（Object）：房子
- 实例（Instance），对象的另外一种叫法。今后“对象”与“实例”两词将不加区分的同义互换使用。

创建类与对象

- 有了类才能有对象
- 类定义中，包含了对象所需的实例属性和实例方法
- 实例属性：类定义中与实例相关联的变量，用来表示一个实例的状态。
    - 例如：一个圆Circle对象，具有属性radius，表示圆的属性：半径。
- 实例方法：类定义中与实例相关联的函数，来定义的一个对象的行为。通过调用对象上的方法，可以让对象完成某个动作。
    - 例如：可以使用get_area()返回圆的面积。get_perimeter()返回周长。

定义类

```
class ClassName:
    initializer
    methods
```

例：一个圆的类

```
import math

class Circle:

    def __init__(self, radius = 1):
        self.radius = radius

    def get_perimeter(self):
        return 2 * self.radius * math.pi

    def get_area(self):
        return self.radius * self.radius * math.pi
```

初始化方法

- 类有一个特殊的方法：`__init__()`，这个方法被称为初始化程序。
    - 二个下划线（double underscores）习惯上可被读作 dunder （Double UNDERscores）
    - `__init__()`，读作：dunder init dunder 或 dunder init

`self` 参数

- 类的方法与普通的函数只有一个特别的区别——它们必须有一个额外的第一个参数名称, 按照惯例它的名称是 self。
- self 代表类的实例，而非类。self 是指向对象本身的参数。
- 可以使用 self 访问在类定义中的对象成员变量和方法。
- 一旦一个实例变量被创建，那么它的作用域就是整个类。

使用类创建对象

- 实例化类其他编程语言中一般用关键字 new，但是在 Python 中并没有这个关键字，类的实例化类似函数调用方式。
- 以下使用类的名称 Circle 来实例化，并通过 **init** 方法接收参数。
- 可以使用点号 . 来访问对象的属性。

```
def main():
    circle1 = Circle()
    circle2 = Circle(5)
    print(circle1.get_perimeter())
    print(circle1.get_area())
    print(circle2.get_perimeter())
    print(circle2.get_area())
    circle2.radius = 10
    print(circle2.get_perimeter())
    print(circle2.get_area())
    print(circle2.radius)

main()
```

```
6.283185307179586
3.141592653589793
31.41592653589793
78.53981633974483
62.83185307179586
314.1592653589793
10
```

访问控制

隐藏属性

- 直接访问对象的属性不是一个好方法，可能会造成异常数据，例如将圆的半径设置为负值。
- 数据隐藏：私有属性是以两个下划线开头
- 同样的，以两个下划线开头的方法是私有方法
- 私有属性和私有方法只能在类内部被访问，不能在类外被访问。

例：数据隐藏后的圆

```
import math

class Circle:

    def __init__(self, radius = 1):
        if radius < 0:
            raise RuntimeError("Negative radius")
        else:
            self.__radius = radius

    def get_radius(self):
        return self.__radius

    def set_radius(self, radius):
        if radius < 0:
            raise RuntimeError("Negative radius")
        else:
            self.__radius = radius

    def get_perimeter(self):
        return 2 * self.__radius * math.pi

    def get_area(self):
        return self.__radius * self.__radius * math.pi
```

使用隐藏属性后的圆的实例

```
def main():
    circle1 = Circle()
    circle2 = Circle(5)
    print(circle1.get_perimeter())
    print(circle1.get_area())
    print(circle2.get_perimeter())
    print(circle2.get_area())
    circle2.set_radius(10)
    # 不可以 circle2.radius = 10，不会影响到半径的值
    # 也不可以 circle2.__radius = 10，会抛异常
    print(circle2.get_perimeter())
    print(circle2.get_area())
```

```
    print(circle2.get_radius())
    # 不可以 print(circle2.radius)
    # 也不可以 print(circle2.__radius)
    circle2.set_radius(-10) # 抛出异常

main()
```

奇怪的知识

- 注意：`__radius`是私有属性，但是`__radius__`并不是。也就是说，私有属性以二个下划线开头，且不能以下划线结尾。首尾都是二个下划线的是系统保留的名称。
- 私有属性并非绝对不能在类外访问，事实上，使用`实例名._类名__私有属性`的名称是可以访问到私有属性的，如`circle1._Circle__radius`，但这种方法是不建议使用的。

属性（property）

- 与实例属性（attribute）不同，此处介绍的属性（property）是通过一个装饰器（@property）将一个方法转换而来的，从而实现用于计算的属性。将方法转换为属性后，可以直接通过方法名来访问方法，而不需要再添加一对小括号。
- 对属性（property）的修改，是通过一个装饰器（@???.setter）将修改函数转换而来的，此处的???要替换为要修改的属性（property）的名字，而且修改函数也必须以属性（property）名来命名。

属性（property）示例

```
import math

class Circle:

    def __init__(self, radius = 1):
        if radius < 0:
            raise RuntimeError("Negative radius")
        else:
            self.__radius = radius

    @property
    def radius(self):
        return self.__radius

    @radius.setter
    def radius(self, radius):
        if radius < 0:
            raise RuntimeError("Negative radius")
        else:
            self.__radius = radius

    @property
    def area(self):
        return self.__radius * self.__radius * math.pi

    @property
    def perimeter(self):
```

```
        return 2 * self.__radius * math.pi

def main():
    circle1 = Circle()
    circle1.radius = 10
    print(circle1.perimeter)
    print(circle1.area)
    print(circle1.radius)

main()
```

类属性与类方法和静态方法

- 实例属性只属于实例自己
- 有的信息不希望每个对象都拥有，例如同一类型对象总数
- 类属性：只关联到类本身的属性
- 类方法：只关联到类本身的方法
- 静态方法：只关联到类本身的方法，与类方法在写法上略有不同

类属性

- 类属性直接定义在类中，只与类相关，不与实例相关
- 类属性的定义语句只执行一次
- 在第一个对象创建之前就存在
- 类属性前不加self
- 使用 `类名.类属性名` 进行访问。

```
class Circle:
    count = 0

    def __init__(self, radius = 1):
        self.__radius = radius
        Circle.count += 1

def main():
    circle1 = Circle()
    circle2 = Circle(5)
    print(Circle.count) # 显示结果为2

main()
```

类属性的使用

- 可以使用 `类名.类属性名` 访问类属性。可以读取，也可以修改。
- 也可以使用 `实例名.类属性名` 访问类属性，但只能读取，不能修改。
- 如果尝试使用 `实例名.类属性名` 修改其值，会导致产生新的实例属性。

```
def main():
    circle1 = Circle()
    circle2 = Circle(5)
    print(Circle.count,circle1.count,circle2.count) # 显示为2 2 2
    Circle.count = 6
    circle1.count = 8
    print(Circle.count,circle1.count,circle2.count) # 显示为6 8 6

main()
```

类方法和静态方法

- 类方法是指使用 @classmethod 修饰的方法，其第一个参数 cls 是类本身，而不是类的实例对象。
- 静态方法是指使用 @staticmethod 修饰的方法，与类方法不同的是，它没有类方法中的第一个类参数。
- 类方法和静态方法一样，都只能访问类属性，不能访问实例属性（因为没有self参数）。在访问类属性时，类方法可以使用 cls 参数做前缀，而静态方法只能使用完整的类名做前缀。
- 类方法和静态方法一样，都不能调用实例方法（因为没有self参数）。类方法与静态方法之间可以互相调用。
- 类方法和静态方法一样，既可以直接通过类名调用，也可以通过类的实例对象调用。

```
class Circle:
    count = 0

    def __init__(self, radius = 1):
        self.__radius = radius
        Circle.count += 1

    @classmethod
    def get_count1(cls):
        return cls.count

    @staticmethod
    def get_count2():
        return Circle.count

def main():
    circle1 = Circle()
    circle2 = Circle(5)
    print(Circle.get_count1())  # 显示为 2
    print(Circle.get_count2())  # 显示为 2
    print(circle1.get_count1()) # 显示为 2
    print(circle1.get_count2()) # 显示为 2

main()
```

常用内置方法和运算符重载

- Python类中提供了很多内置方法，它们的形式为 `__方法名__()` ，如初始化函数 `__init__()` 。
- 与 `__init__()` 类似的还有 `__del__()` ，它将在对象被销毁时自动执行。

- 下面仅举例说明部分内置方法和运算符重载的使用。其它方法可参见下面的表格和示例。

`__str__()`

- `__str__()` 在需要时将对象转换为字符串返回。应用场景如将对象与字符串相加、print(对象)或者str(对象)

```
import math

class Circle:

    def __init__(self, radius = 1):
        self.__radius = radius

    def get_perimeter(self):
        return 2 * self.__radius * math.pi

    def get_area(self):
        return self.__radius * self.__radius * math.pi

    def __str__(self):
        return "Circle with radius " + str(self.__radius) + " has perimeter of " +
str(self.get_perimeter()) + " and area of " + str(self.get_area())

def main():
    circle1 = Circle()
    circle2 = Circle(5)
    print(circle1)
    print(circle2)

main()
```

`__eq__()`

- 判断两个对象是否相等

```
class Circle:
......
    def __eq__(self,other):
        return self.__radius == other.__radius

......
def main():
    circle1 = Circle()
    circle2 = Circle(1)
    if circle1 == circle2:
        print("They are same.")
```

常用运算符重载表

运算符	方法	描述
+	`__add__(self,other)`	加法
-	`__sub__(self,other)`	减法
*	`__mul__(self,other)`	乘法
/	`__truediv__(self,other)`	算术除
//	`__floordiv__(self,other)`	整数除
%	`__mod__(self,other)`	取余
**	`__pow__(self,other)`	幂运算
<	`__lt__(self,other)`	小于
<=	`__le__(self,other)`	小于等于
>	`__gt__(self,other)`	大于
>=	`__ge__(self,other)`	大于等于
==	`__eq__(self,other)`	等于
!=	`__ne__(self,other)`	不等于
[index]	`__getitem__(self,index)`	索引
in	`__contains__(self,value)`	包含
len	`__len__(self)`	求长度
str	`__str__(self)`	转字符串

类的继承

面向对象的编程带来的主要好处之一是代码的重用，实现这种重用的方法之一是通过继承机制。

通过继承创建的新类称为**子类**或**派生类**，被继承的类称为**基类**、**父类**或**超类**。

继承语法

```
class 派生类名(基类名)
    ...
```

在python中继承中的一些特点：

- 1、如果在子类中需要父类的构造方法就需要显式的调用父类的构造方法，或者不重写父类的构造方法。
    - 情况一：**子类需要自动调用父类的方法：**子类不重写**init**()方法，实例化子类后，会自动调用父类的**init**()的方法。

- 情况二：**子类不需要自动调用父类的方法：**子类重写**init**()方法，实例化子类后，将不会自动调用父类的**init**()的方法。
- 情况三：**子类重写init()方法又需要调用父类的方法：**
  - 可以使用 **super** 关键字：

```
super(子类，self).__init__(参数1，参数2，....)
```

  - 还有一种经典写法：

```
父类名称.__init__(self,参数1，参数2，...)
```

- 2、在调用基类的方法时，需要加上基类的类名前缀，且需要带上 self 参数变量。区别在于类中调用普通函数时并不需要带上 self 参数
- 3、Python 总是首先查找对应类型的方法，如果它不能在派生类中找到对应的方法，它才开始到基类中逐个查找。（先在本类中查找调用的方法，找不到才去基类中找）。

如果在继承元组中列了一个以上的类，那么它就被称作"多重继承" 。

语法：

派生类的声明，与他们的父类类似，继承的基类列表跟在类名之后，如下所示：

```
class SubClassName (ParentClass1[, ParentClass2, ...]):
    ...
```

示例：

```
class Parent:        # 定义父类
   parentAttr = 100
   def __init__(self):
      print("调用父类构造函数")

   def parentMethod(self):
      print('调用父类方法')

   def setAttr(self, attr):
      Parent.parentAttr = attr

   def getAttr(self):
      print("父类属性 :", Parent.parentAttr)

class Child(Parent): # 定义子类
   def __init__(self):
      print("调用子类构造方法")

   def childMethod(self):
      print('调用子类方法')

c = Child()          # 实例化子类
c.childMethod()      # 调用子类的方法
c.parentMethod()     # 调用父类方法
```

```
c.setAttr(200)        # 再次调用父类的方法 - 设置属性值
c.getAttr()           # 再次调用父类的方法 - 获取属性值
```

执行结果如下：

```
调用子类构造方法
调用子类方法
调用父类方法
父类属性 : 200
```

方法重写

如果你的父类方法的功能不能满足你的需求，你可以在子类重写你父类的方法：

```
class Parent:        # 定义父类
   def myMethod(self):
      print('调用父类方法')

class Child(Parent): # 定义子类
   def myMethod(self):
      print('调用子类方法')

c = Child()          # 子类实例
c.myMethod()         # 子类调用重写方法
```

执行结果如下：

```
调用子类方法
```

示例1：分数类（MyFraction）

编写一个类 MyFraction，可以存储分数。具体要求如下：

1. 存储时应分别存储分子（numerator）和分母（denominator）。
2. 存储时应该是约分过的最简分数，如10/12应该存储为5/6。提示：正分数约分为最简分数时，分子分母同时除以它们的最大公约数即可。
3. 分数为负值时，则负号标在分子上，如-3/5，分子存储为-3和分母为5。
4. 分数值为0时，分子存储为0，分母存储为1。
5. 分数值是个整数时，分母存储为1。
6. MyFraction类支持使用（分子，分母）方式完成对象创建，存储分数的最简形式；也支持使用（分子）方式完成创建，此时分母取默认值1；也支持使用无参的方式完成对象创建，此时默认分数值为0 。
7. MyFraction类为不可变类，一旦创建便不可修改字段。设置属性（property）字段numerator，denominator供外部访问，为只读属性。
8. MyDate类支持分数之间的四则运算，即对 +，-，*，/ 运算符进行重载
9. MyDate类支持比较大小，即对 <, <=, >, >=, ==, != 运算符进行重载
10. MyFraction类支持转字符串，重载 **str**() 实现转字符串，形式为“分子/分母” 。

11. MyFraction类支持实现用[0]和[1]下标取分数的分子和分母。
12. 增加必要的错误异常处理逻辑。

```
class MyFraction:

    @staticmethod
    def reduce(numerator, denominator):
        if numerator*denominator< 0:
            sign = -1
        else:
            sign = 1
        gcd = 1
        numerator, denominator = abs(numerator),abs(denominator)
        for i in range(min(numerator, denominator),0,-1):
            if numerator%i==0 and denominator%i==0:
                gcd = i
                break
        numerator, denominator = sign*numerator//gcd, denominator//gcd if numerator!=0 else
1
        return numerator, denominator


    def __init__(self, numerator=0, denominator=1):
        ''' init the instance '''
        if denominator==0:
            raise ZeroDivisionError("division by zero")
        self.__numerator, self.__denominator = self.reduce(numerator, denominator)

    def __str__(self):
        ''' convert to string '''
        return str(self.__numerator)+"/"+str(self.__denominator)

    @property
    def numerator(self):
        return self.__numerator

    @property
    def denominator(self):
        return self.__denominator

    def __eq__(self,other):
        ''' equal '''
        return self.__numerator == other.__numerator and self.__denominator ==
other.__denominator

    def __ne__(self,other):
        '''not equal'''
        return not self.__eq__(other)

    def __lt__(self,other):
        '''less than'''
        return self.__numerator*other.__denominator < other.__numerator*self.__denominator
```

```
    def __le__(self,other):
        '''less than or equal'''
        return self.__lt__(other) or self.__eq__(other)

    def __gt__(self,other):
        '''greater than'''
        return other.__lt__(self)

    def __ge__(self,other):
        '''greater than or equal'''
        return not self.__lt__(other)

    def __add__(self,other):
        x = self.__numerator*other.__denominator + other.__numerator*self.__denominator
        y = self.__denominator * other.__denominator
        return MyFraction(x,y)

    def __sub__(self,other):
        x = self.__numerator*other.__denominator - other.__numerator*self.__denominator
        y = self.__denominator * other.__denominator
        return MyFraction(x,y)

    def __mul__(self,other):
        x = self.__numerator * other.__numerator
        y = self.__denominator * other.__denominator
        return MyFraction(x,y)

    def __truediv__(self,other):
        x = self.__numerator * other.__denominator
        y = self.__denominator * other.__numerator
        return MyFraction(x,y)

    def __getitem__(self,index):
        ''' get by index '''
        if index == 0:
            return self.__numerator
        elif index == 1:
            return self.__denominator
        else:
            raise IndexError("Index out of range.")

def main():
    f1 = MyFraction(7, 8);
    f2 = MyFraction(-5,12);
    assert f1+f2==MyFraction(11,24)
    assert f1-f2==MyFraction(31,24)
    assert f1*f2==MyFraction(-35,96)
    assert f1/f2==MyFraction(-21,10)
    assert f1>f2

    print("Congratulations! All tests are passed!")
```

```
if __name__=="__main__":
    main()
```

示例2：时间类（MyTime）

编写一个类 MyTime ，可以存储时间。具体要求如下：

1. 设 00:00:00 为时间轴的原点。在类中仅需保存一个私有字段__second，表示该时间在时间轴的坐标。坐标单位为秒，坐标均为非负数。
2. MyTime类支持使用（时，分，秒）方式完成对象创建，也支持使用（seconds=坐标）的方式完成对象创建，如时分秒和坐标均指定，以坐标为准，如时分秒和坐标均省略，则设置为坐标为0的对象。
3. MyTime类为不可变类，一旦创建便不可修改__seconds字段。设置属性（property）字段 hour, minute, second, seconds 供外部访问，为只读属性。
4. MyTime类支持使用下标运算符获得时分秒，即MyTime()[0]、MyTime()[1]、MyTime()[2]分别可以获得此实例的时、分、秒，为只读访问。
5. MyTime类支持转字符串，转出的字符串格式为hh:mm:ss，时2位、分2位、秒2位，不足位的用0补齐，如09:40:05 等。
6. MyTime类支持比较大小，实例比较的结果符合坐标比较的结果。
7. MyTime类支持两个实例做减法，相减的值为两个实例坐标之差。
8. MyTime类的实例支持加上一个整数（可正可负），返回一个新的MyTime实例，新实例的坐标是原实例坐标加上此整数而成的。注意加负整数时要写成MyTime()+(-1)，不能写成MyTime()-1，否则会与上一条的两实例做减法相混淆。
9. 增加必要的错误异常处理逻辑。

```
class MyTime:

    @staticmethod
    def time_to_seconds(hour, minute, second):
        if not 0 <= hour < 24:
            raise ValueError("Hour value is out of range")
        elif not 0 <= minute < 60:
            raise ValueError("Minute value is out of range")
        elif not 0 <= second < 60:
            raise ValueError("Second value is out of range")
        else:
            return hour * 60 * 60 + minute * 60 + second

    @staticmethod
    def seconds_to_time(seconds):
        if not 0 <= seconds < 60 * 60 * 24:
            raise ValueError("Seconds value is out of range")
        hour = seconds // ( 60 * 60 )
        minute = seconds % ( 60 * 60 ) // 60
        second = seconds % 60
        return hour, minute, second

    def __init__(self, hour=0, minute=0, second=0):
```

```
        ''' init the instance '''
        self.__seconds = self.time_to_seconds(hour, minute, second)

    def __str__(self):
        ''' convert to string '''
        time = self.seconds_to_time(self.__seconds)
        s_hour = "{:0>2d}".format(time[0])
        s_minute = "{:0>2d}".format(time[1])
        s_second = "{:0>2d}".format(time[2])
        return s_hour+":"+s_minute+":"+s_second

    @property
    def seconds(self):
        return self.__seconds

    @property
    def time(self):
        return self.seconds_to_time(self.__seconds)

    def __eq__(self,other):
        ''' equal '''
        return self.__seconds == other.__seconds

    def __ne__(self,other):
        '''not equal'''
        return not self.__eq__(other)

    def __lt__(self,other):
        '''less than'''
        return self.__seconds < other.__seconds

    def __le__(self,other):
        '''less than or equal'''
        return self.__lt__(other) or self.__eq__(other)

    def __gt__(self,other):
        '''greater than'''
        return other.__lt__(self)

    def __ge__(self,other):
        '''greater than or equal'''
        return not self.__lt__(other)

    def __add__(self,seconds):
        ''' add an integer as seconds '''
        new_seconds = self.__seconds + seconds
        new_time = self.seconds_to_time(new_seconds)
        return MyTime(*new_time)

    def __sub__(self,other):
        ''' get distance of two instances '''
        return self.__seconds - other.__seconds
```

```
    def __getitem__(self,index):
        ''' get hour,minute,second by index '''
        if index > 2:
            raise IndexError("Index out of range. 0..2 for hour, minute, second")
        time = self.seconds_to_time(self.__seconds)
        return time[index]

    def __len__(self):
        ''' get seconds of the instance '''
        return self.__seconds

def main():
    t0 = MyTime()
    t1 = t0 + 3600
    t2 = t1 + 60 * 15
    assert t2 == MyTime(1,15,0)

    print("Congratulations! All tests are passed!")

if __name__=="__main__":
    main()
```

这个实现展示了Python面向对象编程的主要特性，包括封装、运算符重载、类方法和静态方法等。